FRAUDSHIELD

Intelligent Fraud Detection
and Risk Analysis System

MACHINE LEARNING PROJECT GUIDE

A practical blueprint for data preparation, model training, evaluation, API development, dashboard design, testing, and deployment.

Project category: Supervised Machine Learning • Classification • Risk Analytics

Suggested level: Final-year B.Tech CSE / portfolio project

Document purpose: Planning and implementation reference. Model performance must be measured after training; no results are assumed in this guide.

Prepared as a project blueprint • October 2026

Contents

- 1. Executive Summary
- 2. Problem Statement and Objectives
- 3. Scope, Users, and Expected Output
- 4. Technology Stack
- 5. Dataset and Data Dictionary
- 6. End-to-End Machine Learning Workflow
- 7. Model Selection and Evaluation
- 8. System Architecture
- 9. Application Features and Dashboard
- 10. Project Folder Structure
- 11. Step-by-Step Implementation Plan
- 12. API Design
- 13. Testing, Security, and Responsible Use
- 14. Deployment Plan
- 15. Timeline and Deliverables
- 16. Challenges and Future Enhancements
- 17. Resume Description and Viva Questions
- 18. Learning Resources

Project at a glance

Item	Plan
Primary task	Binary classification: legitimate vs. potentially fraudulent transaction
Starter dataset	Public European credit-card transactions dataset (anonymized features)
Baseline model	Logistic Regression
Comparison models	Random Forest; optionally XGBoost if installed and justified
Core metrics	Precision, recall, F1-score, PR-AUC; confusion matrix
Demo interface	Streamlit first; optional React dashboard later
Optional service layer	FastAPI for model inference
Storage	CSV for prototype; SQLite/MongoDB for history if needed

1. Executive Summary

FraudShield is a machine learning project that estimates whether a financial transaction resembles patterns associated with fraud. It uses historical, labeled transaction data to train a classification model, evaluates the model on unseen data, and presents predictions and performance indicators in an interactive dashboard. The system is intended as a learning and decision-support prototype—not as a production banking control or an automatic declaration that a person has committed fraud.

The project demonstrates the full machine learning lifecycle: defining a problem, exploring data, cleaning and transforming features, splitting data, training models, handling class imbalance, evaluating errors, saving the trained pipeline, exposing predictions, and documenting limitations.

2. Problem Statement and Objectives

Problem statement: Fraudulent transactions are relatively rare in many transaction datasets. A model that labels every transaction as legitimate may appear highly accurate while failing to detect fraud. FraudShield investigates how machine learning can identify higher-risk patterns while measuring both missed fraud and false alarms.

- **Objective 1:** Explore and understand transaction data and label distribution.
- **Objective 2:** Build a reproducible preprocessing and training pipeline.
- **Objective 3:** Compare a simple baseline with at least one tree-based model.
- **Objective 4:** Evaluate precision, recall, F1-score, PR-AUC, and the confusion matrix.
- **Objective 5:** Build an interface to score sample transactions and visualize model performance.
- **Objective 6:** Explain limitations, data privacy concerns, and safe use of predictions.

3. Scope, Users, and Expected Output

Intended users: students, educators, and project evaluators exploring fraud detection methods. A real financial institution would need stronger validation, access controls, monitoring, and compliance reviews.

Inputs: transaction attributes available in the selected dataset. **Outputs:** a predicted class and, if calibrated and validated, a risk score. The risk score is a model estimate—not proof of fraud.

Recommended minimum viable product (MVP)

- A notebook documenting data exploration and preprocessing.
- A training script that saves the complete preprocessing-plus-model pipeline.
- A model evaluation report with metrics and a confusion matrix.
- A simple Streamlit app that accepts a dataset row or selected sample and shows a prediction.
- A README with setup instructions, dataset attribution, limitations, and screenshots.

Out of scope for the first version

- Connecting to real bank accounts or payment networks.
- Making automatic account freezes or accusing a person of fraud.
- Claiming real-world detection performance from a small public benchmark alone.

4. Technology Stack

Technology	Purpose	Required?
Python	Main language for analysis, training, and inference	Yes
Pandas / NumPy	Load, inspect, clean, and transform tabular data	Yes
Matplotlib / Seaborn	EDA charts and confusion matrix visualization	Recommended
Scikit-learn	Splitting, preprocessing, models, metrics, pipelines	Yes
Imbalanced-learn	Optional resampling such as SMOTE inside training folds	Optional
XGBoost	Optional gradient-boosted tree comparison	Optional
Jupyter Notebook	Exploration and experimentation	Recommended
Streamlit	Fast interactive demo dashboard	Recommended for MVP
FastAPI	Serve predictions through a REST API	Optional advanced step
React + Node.js/Express	Custom full-stack user interface and backend	Optional advanced step
SQLite / MongoDB	Store prediction history and app metadata	Optional
Git + GitHub	Version control and project portfolio	Recommended

Recommended build order

Start with Python, Pandas, Scikit-learn, and Streamlit. This keeps the first version focused on learning machine learning correctly. Add FastAPI and then a React/Node.js interface only after the model pipeline works reliably. A complex web stack cannot compensate for data leakage or weak model evaluation.

5. Dataset and Data Dictionary

Suggested starter dataset: the public European credit-card fraud dataset hosted on Kaggle under the MLG-ULB dataset listing. It contains anonymized numerical features, transaction amount, elapsed time, and a target label. Check the dataset page for its current description, terms, and download instructions.

Dataset link: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

Field	Meaning (starter dataset)
Time	Elapsed time from the first recorded transaction in the dataset, in seconds
V1–V28	Anonymized numerical features produced by a transformation of original variables
Amount	Transaction amount in the dataset's stated units
Class	Target label: typically 0 = non-fraud and 1 = fraud for this dataset

Important dataset notes

- Confirm the target label mapping and column names after downloading.
- The dataset's anonymized features limit direct business interpretation.
- Its class distribution is highly imbalanced; always compare class counts.
- Do not use the test set to choose features, tune parameters, or set a threshold.
- Avoid publishing raw transaction records or sensitive data in screenshots or repositories.

6. End-to-End Machine Learning Workflow

Step 1 — Define the prediction task

Predict whether a transaction is labeled fraudulent (1) or legitimate (0). Decide how the system should balance missed fraud against false alerts.

Step 2 — Load and inspect data

Read the CSV with Pandas. Check shape, data types, missing values, duplicate rows, target counts, and summary statistics.

Step 3 — Perform EDA

Plot the target distribution, amount distributions, correlations where meaningful, and examples of class imbalance. Document observations.

Step 4 — Create a train/test split

Use a reproducible split and stratify by the target for an initial benchmark. If timestamps and future deployment matter, consider a chronological split instead.

Step 5 — Build preprocessing

Apply transformations using a Scikit-learn Pipeline/ColumnTransformer so preprocessing is learned from training data only.

Step 6 — Train a baseline

Fit Logistic Regression with appropriate class weighting and evaluate it before attempting complex models.

Step 7 — Train comparison model(s)

Try Random Forest or a gradient-boosting model. Tune only with training data and cross-validation.

Step 8 — Address imbalance carefully

Use class weights or training-fold resampling. Never resample the full dataset before splitting, because this can leak information.

Step 9 — Evaluate errors

Review precision, recall, F1-score, PR-AUC, ROC-AUC, confusion matrix, and the false-positive/false-negative trade-off.

Step 10 — Select a decision threshold

Choose a threshold on validation data based on the project's stated error costs. Do not assume 0.5 is automatically best.

Step 11 — Save the full pipeline

Persist preprocessing and the estimator together. Record library versions and the dataset/model version.

Step 12 — Build the interface

Load the saved pipeline, validate inputs, show the prediction and model limitations, and display evaluation charts.

Step 13 — Test and document

Test normal, edge, malformed, and missing inputs. Write setup steps, data provenance, limitations, and measured results.

7. Model Selection and Evaluation

Recommended models

Model	Why use it	Watch-outs
Logistic Regression	Interpretable baseline; fast; useful probability baseline	May miss nonlinear relationships; scale features when appropriate
Random Forest	Captures nonlinear patterns and feature interactions	Can be computationally heavier; probability scores may need calibration
XGBoost (optional)	Strong gradient-boosted model for many tabular tasks	Extra dependency and tuning; can overfit if poorly controlled
Isolation Forest (optional)	Explores anomaly detection when labels are unavailable or incomplete	Anomaly is not the same as confirmed fraud; evaluate separately

Metrics you should understand

- **Precision:** Of transactions flagged as fraud, how many are actually fraud in the labeled test set?
- **Recall:** Of all labeled fraudulent transactions, how many did the model identify?
- **F1-score:** Harmonic mean of precision and recall.
- **PR-AUC / Average Precision:** Useful for assessing ranking quality when the positive class is rare.
- **ROC-AUC:** Measures ranking across thresholds; do not use it alone for heavily imbalanced data.
- **Confusion matrix:** Shows true positives, false positives, true negatives, and false negatives.
- **Calibration:** Checks whether predicted probabilities correspond to observed frequencies.

Accuracy warning: if only a tiny fraction of transactions are fraudulent, a model can achieve very high accuracy by predicting the majority class for every row. Report minority-class metrics and the confusion matrix.

Leakage prevention checklist

- Split data before fitting preprocessing steps or performing resampling.
- Fit scalers, imputers, feature selection, and resampling only within training folds.
- Use validation data for tuning and threshold selection; reserve the test set for final evaluation.
- If duplicate or related transactions can appear across splits, use a split strategy that prevents overly optimistic results.
- If the deployment setting is future transactions, consider time-based evaluation.

8. System Architecture

MVP architecture (recommended)

CSV dataset → Pandas/Scikit-learn training pipeline → saved model pipeline → Streamlit app → user-facing prediction and evaluation dashboard.

Advanced architecture (optional)

React UI → Node.js/Express application API → Python FastAPI inference service → saved ML pipeline. MongoDB or SQLite may store non-sensitive demo history and metadata. Keep secrets in environment variables, validate inputs on the server, and never expose private data in logs.

Prediction flow

- User submits transaction features or selects a sample record.

- The app validates the required fields and data types.
- The saved pipeline transforms the inputs and produces a prediction score.
- The app applies the selected decision threshold and displays the result.
- The interface explains that the score is an estimate and may be wrong.
- Optional: save a minimal audit record without storing unnecessary sensitive information.

9. Application Features and Dashboard

Recommended dashboard pages

- **Overview:** dataset size, fraud prevalence, and model version.
- **Transaction checker:** input validation, predicted class, and risk score if justified.
- **Model performance:** confusion matrix, precision, recall, F1-score, PR curve, and ROC curve.
- **Data explorer:** class distribution, amount summaries, and selected feature plots.
- **Model comparison:** compare candidates using the same held-out evaluation setup.
- **About and limitations:** dataset source, label definitions, known limitations, and responsible-use note.

UX guidance

- Use clear labels such as 'Predicted high risk' instead of 'Fraud confirmed'.
- Show which model version generated the result and the threshold used.
- Explain false positives and false negatives in plain language.
- Do not present a risk score as a calibrated probability unless calibration has been tested.
- Ensure the app handles invalid inputs gracefully and is usable on smaller screens.

10. Project Folder Structure

```
FraudShield/ ■■■ data/ ■ ■■■ raw/ # Keep downloaded data local; do not commit it ■ ■■■
processed/ # Optional generated data, usually ignored by Git ■■■ notebooks/ ■ ■■■
01_exploratory_analysis.ipynb ■■■ src/ ■ ■■■ data_validation.py ■ ■■■ preprocessing.py ■ ■■■
train.py ■ ■■■ evaluate.py ■ ■■■ predict.py ■■■ models/ ■ ■■■ fraud_pipeline.joblib ■■■ app/
■ ■■■ streamlit_app.py ■■■ tests/ ■ ■■■ test_preprocessing.py ■ ■■■ test_prediction.py ■■■
reports/ ■ ■■■ figures/ ■ ■■■ model_evaluation.md ■■■ requirements.txt ■■■ .gitignore ■■■
README.md
```

Keep raw datasets and generated model files out of Git when large or subject to dataset terms. Never commit API keys, credentials, or personal financial data.

11. Step-by-Step Implementation Plan

1. Set up the environment

Install a supported Python version and create a virtual environment. Install Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn, Joblib, and Streamlit. Keep a requirements file.

2. Download and inspect the dataset

Download the dataset from its official listing, place the CSV under data/raw, and verify the label mapping and license/terms.

3. Explore the data

Create an EDA notebook. Inspect class counts, missing values, duplicate rows, numeric ranges, and feature distributions.

4. Define a reproducible split

Separate features and target. Use stratification for a first benchmark or a time-based split if simulating future deployment.

5. Train the baseline

Use a Scikit-learn Pipeline with appropriate preprocessing and Logistic Regression. Set a fixed random seed where relevant.

6. Compare a second model

Train Random Forest or another justified model with the same split and evaluation metrics.

7. Tune responsibly

Use cross-validation on the training portion. Try class weights first; if using SMOTE, place it inside an imbalanced-learn pipeline and apply it only to training folds.

8. Evaluate the selected model

Produce a classification report, confusion matrix, PR curve, PR-AUC, and threshold analysis on validation/test data as appropriate.

9. Save the complete pipeline

Use joblib to persist preprocessing plus estimator together. Pin compatible dependency versions and document model provenance.

10. Build Streamlit interface

Create a page to select a sample or enter features, load the saved pipeline, validate input, and show results and limitations.

11. Test and improve

Test malformed and boundary inputs, ensure repeatable predictions, and verify that the interface uses the same feature order as training.

12. Document and publish

Write README instructions, add screenshots and a model card, push code to GitHub, and deploy only after checking data and secrets are excluded.

Example environment setup

```
python -m venv .venv
Windows: .venv\Scripts\activate
macOS/Linux: source .venv/bin/activate
python -m pip install pandas numpy scikit-learn matplotlib seaborn joblib streamlit jupyter
```

12. API Design (Optional Advanced Version)

Endpoint	Method	Purpose
/health	GET	Confirm the inference service is running
/predict	POST	Validate input features and return predicted class and score
/model-info	GET	Return model version, feature schema, and threshold metadata
/metrics	GET	Return precomputed evaluation metrics; do not expose private records

Example request/response (illustrative schema only)

```
Request: { "features": { "Time": 1000, "Amount": 125.50, "V1": 0.1, ... } }
Response: { "prediction": 0, "label": "legitimate-like", "score": 0.03, "model_version": "1.0" }
```

The example values are placeholders, not a trained model's output. Validate the exact feature schema and label mapping for your dataset. Avoid using the term 'probability' unless calibration is assessed.

13. Testing, Security, and Responsible Use

Testing checklist

- Data schema checks: required columns exist and have expected types.
- Preprocessing checks: no unexpected missing or infinite values.
- Pipeline consistency: same columns and order at training and inference time.
- Model checks: output shape and class labels are valid.
- API checks: invalid payloads return safe error messages.
- Reproducibility: document versions, seeds, split strategy, and model artifacts.
- UI checks: distinguish predicted labels from confirmed facts.

Security and privacy

- Use only public or properly authorized data in the student prototype.
- Do not collect bank credentials, card numbers, CVVs, or other payment secrets.
- Use environment variables for credentials and keep .env files out of version control.
- Limit logging; do not log raw transaction details unless necessary and authorized.
- Restrict access to the inference API and use HTTPS when deployed.
- Review dependency updates and protect model artifacts from untrusted uploads.

Responsible use

A false positive may inconvenience a legitimate customer; a false negative may miss fraud. Any real-world system requires human review, clear escalation rules, monitoring for data drift, privacy safeguards, and fairness and security review. The prototype must not be used as the sole basis for denying services or accusing individuals.

14. Deployment Plan

Begin locally

- Run the notebook and training script locally.
- Confirm the saved model can be loaded in a clean environment.
- Run the Streamlit app and test several records from a held-out sample.
- Confirm the README reproduces the setup from scratch.

Deployment options

Option	Best for	Notes
Streamlit Community Cloud	Simple Python demo	Check current limits, repository requirements, and availability before deployment
FastAPI on a cloud host	Standalone inference API	Secure endpoints, set resource limits, and version the model
React + Node.js + FastAPI	Full-stack portfolio version	More integration work; build after the ML pipeline is stable

Before publishing

- Remove raw datasets if redistribution is not permitted.
- Remove secrets, private environment files, and personal data.
- Add a clear demo-only disclaimer.
- Include actual test metrics and the evaluation split used.
- Record known limitations and how to reproduce the result.

15. Timeline and Deliverables

Week / Days	Work	Deliverable
Days 1–2	Dataset, problem definition, EDA	Notebook and data notes
Days 3–4	Preprocessing and baseline model	Reproducible baseline
Days 5–6	Model comparison and validation	Evaluation report
Days 7–8	Threshold analysis, model saving, tests	Saved pipeline and tests
Days 9–10	Streamlit dashboard	Working local demo
Days 11–12	Documentation and deployment	GitHub repository and demo
Optional extra week	FastAPI + React/Node integration	Full-stack version

The schedule is an estimate. Spend more time on evaluation and understanding errors rather than rushing to add extra algorithms.

16. Challenges and Future Enhancements

Likely challenges

- Severe class imbalance can make accuracy misleading.
- Anonymized features are difficult to interpret in business terms.
- Public benchmark data may not represent current transaction behavior.

- A threshold that improves recall may also increase false alarms.
- Data leakage can produce unrealistically strong test metrics.
- Fraud patterns change over time, so deployed models need monitoring.

Future enhancements

- Calibrate risk scores and make threshold selection cost-sensitive.
- Add SHAP-based explanations where compatible with the selected model.
- Evaluate temporal drift and retraining strategies.
- Compare supervised classification with anomaly detection.
- Create a reviewer workflow for investigating flagged records.
- Add model versioning, monitoring, audit logs, and access controls.
- Use authorized, more representative data before drawing real-world conclusions.

17. Resume Description and Viva Questions

Resume-ready description (use only after implementing the features)

FraudShield — Intelligent Fraud Detection and Risk Analysis System
Developed a machine learning prototype to classify financial transactions as legitimate-like or potentially fraudulent using Python, Pandas, and Scikit-learn. Compared classification models using precision, recall, F1-score, PR-AUC, and confusion-matrix analysis. Built an interactive Streamlit dashboard for sample scoring and model evaluation. Documented class-imbalance handling, leakage prevention, and responsible-use limitations.

Do not add unimplemented technologies or claim a performance score until you have measured it.

Viva / interview questions to prepare

What is fraud detection?
A classification or anomaly-detection task that identifies transactions needing further review.

Why is this a classification problem?
The supervised target has categories such as non-fraud and fraud.

What is class imbalance?
One class occurs much less often than the other; fraud is often the minority class.

Why is accuracy insufficient?
A majority-class model may be accurate overall while detecting almost no fraud.

What is precision?
The fraction of predicted-positive cases that are actually positive in labeled evaluation data.

What is recall?
The fraction of actual positive cases detected by the model.

What is a false positive?
A legitimate transaction incorrectly flagged as potentially fraudulent.

What is a false negative?
A fraudulent transaction incorrectly classified as legitimate.

What is data leakage?
Information from outside the training process improperly influences model fitting or evaluation.

Why use a pipeline?
It keeps preprocessing and the estimator together and reduces train/inference inconsistency.

Why might PR-AUC help?
It focuses on precision-recall behavior, which is often informative when positives are rare.

Is a model prediction proof of fraud?
No. It is an estimate that needs context and, in real use, appropriate review.

18. Learning Resources

Resource	URL	Use
Scikit-learn user guide	https://scikit-learn.org/stable/user_guide.html	Pipelines, preprocessing, model selection, metrics, and classification.
Scikit-learn model evaluation	https://scikit-learn.org/stable/modules/model_evaluation.html	Classification metrics and evaluation tools.
Imbalanced-learn documentation	https://imbalanced-learn.org/stable/	Resampling methods and leakage-safe pipelines.
Streamlit documentation	https://docs.streamlit.io/	Build an interactive Python dashboard.
FastAPI documentation	https://fastapi.tiangolo.com/	Optional API layer for model inference.
Kaggle dataset listing	https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud	Starter credit-card fraud dataset.
Pandas documentation	https://pandas.pydata.org/docs/	DataFrames, loading, cleaning, and analysis.

Final checklist before calling the project complete

- I can explain the problem, dataset, labels, and chosen features.
- I can explain the difference between classification, anomaly detection, and a risk score.
- I have a reproducible training pipeline and held-out evaluation.
- I report precision, recall, F1-score, PR-AUC, and the confusion matrix.
- I have prevented data leakage and documented my split strategy.
- The app validates inputs and labels predictions as estimates.
- The GitHub repository includes setup instructions and excludes secrets/private data.
- I can explain the model's errors and limitations during an interview.

FraudShield is a learning prototype. It should not be represented as a production-grade banking fraud system without substantial additional validation, security engineering, monitoring, and compliance work.